Worklive Product Requirements

Product strategy and implementation plan

Version 1.1 • 30 September 2026 • Subscription and CLI requirements added

Recommendation. Build Worklive as a personal AI team that turns a clear brief into a finished, reviewable result. Its 3D office should help the user understand who owns the work, what is happening, and where intervention is needed. The first productive release should complete research and document tasks through the browser already present in the prototype.

Confirmed execution requirement. Every AI interaction runs through the user's official Codex or Claude Code terminal tool and subscription. Each agent has a provider assignment and a matching shirt color. Worklive makes no direct model API calls and offers no automatic API-billing fallback. Section 7 defines the integration boundary and release conditions.

This PRD defines the productive core, the relationship between real work and office life, the requirements for a trustworthy browser, and a staged route from the existing prototype. It also identifies useful concepts from TerminalOS and Workspace OS and compares closely related products.

Initial audience assumption. Start with an individual founder, creator, researcher or project lead who wants help investigating a topic and preparing useful work. Validate this audience before expanding into a software development platform or a general business automation suite.

First success. A user asks for a sourced comparison, watches an agent investigate the sources, receives an editable brief and comparison table, and can request a revision without restating the project. A second agent helps only when its distinct assignment improves the result.

Main product hypothesis. Persistent characters and visible work can make delegation easier to understand and more pleasant to return to. The hypothesis succeeds only if users finish valuable work with less supervision; visual appeal alone does not establish productivity.

1 Product decisions

Decision	Recommendation	Reason
Primary workflow	Research to reviewed document	Fits the browser prototype and produces an outcome users can inspect.
Desktop foundation	Continue Tauri with a browser worker	Preserves working progress and Rust ownership of local state.
Execution	Official local Codex and Claude Code CLIs	Use the user's signed-in tool; no direct model API adapter.
Agent identity	Provider selection and shirt color	Codex teal; Claude terracotta; persistent text labels.
Collaboration	One accountable owner; bounded helpers	Separate sessions and explicit assignments preserve responsibility.
Launch audience	One user on Apple Silicon macOS	Matches the present prototype and limits integration scope.

Decision	Recommendation	Reason
Office life	Optional, local ambient simulation	Retains the playful identity without hidden model activity.
Commercial direction	Validate repeat use before pricing	Demand and willingness to pay remain hypotheses.

Execution through subscriptions and official CLIs, per-agent provider choice and provider shirt colors are confirmed user requirements. Other recommendations remain proposed decisions. The established visual requirements are the 3D office, named agents with roles and personalities, desks created with a plus button, visible browser work, enlarged screens, and a lively shared space. Worklive is the working name; Life remains a naming alternative rather than a separate product.

2 The work Worklive should do

The recurring problem

Delegating work today often creates another conversation to manage. The user has to supply context repeatedly, discover what the agent actually did, recover intermediate files and decide whether a confident answer is usable. Worklive should keep the project, evidence, decisions and deliverables together across multiple runs.

The office provides a spatial overview of a small team. A project provides the durable context. Tasks describe bounded work. Artifacts are the reusable results. Conversation supports those objects and does not replace them.

First jobs to support

Job	Example request	Useful result	Release
Research and compare	Compare three tools against these five requirements.	Sourced matrix, recommendation and limitations.	MVP
Prepare a brief	Turn these notes and sources into a project brief.	Editable document with unresolved questions.	MVP
Review a draft	Check this brief for unsupported claims and gaps.	Comments or a proposed revision with evidence.	MVP
Track a project	Tell me what changed and what needs a decision.	Status brief grounded in project tasks and files.	Next
Monitor a topic	Check these sources each week for relevant changes.	Change report with a quiet no-change outcome.	Next
Operate connected tools	Prepare a CRM update or an email response.	Previewed changes with explicit authority to apply.	Later
Build software	Implement and test this small feature.	Isolated changes, test evidence and review.	Later

Research is a practical starting point because evidence can be collected in the browser and assessed independently of the agent’s narrative. Document preparation builds directly on that evidence.

Repetitive authenticated transactions have more dependencies and should follow after interruption, review and recovery work reliably.

Scope boundaries

The MVP includes one workspace at a time, projects, up to six visible agents, a maximum of two simultaneous model runs, and one live browser tab per running agent. It supports supplied public URLs and browser-based search, selected local text or Markdown inputs, plain-text and Markdown drafts, CSV comparisons, and a basic DOCX export. The productive MVP targets both Codex and Claude Code through separate official CLI adapters. Implement and validate Codex first, then Claude Code against the same contract; both must pass before claiming the full two-provider MVP. Release remains subject to the integration conditions in section 7.

The first release excludes arbitrary agent shell commands beyond the official CLI launch and approved Worklive tools, automatic external sending or publishing, purchases, broad account access, autonomous hiring, full Office editing, a room editor, multiplayer, mobile control and always-on cloud execution. Persistent authenticated browsing and scheduled productive routines are the next stage. Existing cosmetic routines stay available.

The product should remain useful with a single agent. Adding desks must never be a prerequisite for completing a simple task.

3 Comparable products and implications

Research basis. The following comparisons use primary product documentation and repositories accessed on 30 September 2026. Capability descriptions reflect those sources, not a hands-on reliability benchmark. The implications are recommendations for Worklive. Availability and integration support can change.

Visual agent workspaces

Project	Documented approach	Implication for Worklive
Claw3D [S1]	A 3D office and interaction layer connected to external runtimes, with agent management, activity and operational surfaces.	Closest conceptual overlap. A 3D office alone is not a distinct position; focus on accessible knowledge work and trustworthy results.
Pixel Agents [S2]	Maps coding agent activity into animated characters. Ships as a VS Code extension and standalone browser surface. Claude Code is the reference integration.	Ground animation in real events. Keep present integrations separate from the roadmap.
AgentRoom [S3]	Tauri app that watches agent transcripts, shows project offices and supports session search.	A visual observer is useful, but owning execution is a separate responsibility. Worklive needs a durable task model, not just transcript watching.
OpenHands Agent Canvas [S9]	Visual coding workspace with harness connections, parallel worktrees and local or remote backends.	A development edition would need serious isolation and review. Defer competing on the entire coding workflow.

Productive agent systems

Product	Documented approach	Implication for Worklive
Paperclip [S4] [S5]	Organizes agents, tasks, budgets and approvals. Agents execute through discrete heartbeats and retain session continuity.	Borrow explicit ownership and event-driven execution. A small personal team does not need a corporate hierarchy to start.
Claude Cowork [S6]	Knowledge work with parallel subtasks and a built-in browser separate from the user's ordinary browser.	Documents and integrated browsing already set a high baseline. Worklive must improve continuity and supervision, not only add a character.
Manus [S7]	Cloud browsing plus an authorized local browser extension, with visible task tabs and user intervention.	Control ownership and authenticated context are core product concerns. Do not assume an isolated browser inherits existing logins.
OpenClaw [S8]	Distinguishes managed isolated browsing from profiles attached to existing signed-in sessions.	Model browser identity and permission explicitly. A runtime integration is a future option, not a reason to adopt all of its capabilities.
n8n [S10]	Can pause selected AI tool calls for human approval of the proposed parameters.	Make approval an enforceable execution gate and a concrete preview. Mature repeatable workflows may eventually connect through an adapter.

What the evidence changes

Positioning. Worklive should be a calm, persistent workspace for delegating knowledge work to a recognizable team. The defensible product opportunity is the quality of its task lifecycle, evidence, continuity and user control. The visual office contributes orientation and attachment, but competitors already use similar metaphors.

Collaboration. Anthropic reports that parallel research benefits from separate contexts, while coordination and token usage can rise substantially. Its internal results are specific to its system and do not predict Worklive's performance. Worklive should compare one agent against a bounded team on the same tasks before making collaboration the default. [S11]

Architecture. OpenAI distinguishes a manager that calls specialists from a handoff that transfers conversational ownership. Worklive should use the first pattern for its initial shared projects: one owner remains accountable for the result. The official CLIs supply the agent loops; Worklive owns product state, bounded delegation and its tool authority. SDK examples are architectural references only, not a proposed direct API execution path. [S12] [S13]

Validation. There is no evidence yet that the office reduces supervision time or that users will pay for it. Test those hypotheses directly, including a simple list view as a baseline. Do not use repository popularity or competitor marketing claims as market validation.

4 The first complete user journey

Example project

The user creates a project called Customer research tools and asks: “Compare three feedback tools for a five-person product team. Use these vendor links. Check exports, collaboration, integrations and published pricing. Produce a two-page brief and a comparison table. Flag anything you cannot verify.”

1. Worklive proposes a compact task card containing the goal, output formats, source scope and stopping limits. The user can edit it and start immediately. Missing optional preferences become visible assumptions; missing information essential to the answer produces one focused question.
2. Ada owns the task using her selected Codex or Claude Code session. Her shirt and provider label show the active runtime. Her monitor shows the actual page being inspected. The task panel shows completed steps, the current action and source references. It does not invent a completion percentage for open-ended research.
3. If useful and allowed, Ada gives Milo a separate assignment to check export limitations across the same vendors. He uses his own selected CLI, which may differ from Ada’s, and receives only the project material and permissions required for that assignment. The user can see the assignment and its contribution.
4. A source that requires login becomes a visible blocker. The agent can continue independent research, use an alternative permitted source or ask for help. It must not invent inaccessible facts.
5. The user enlarges Ada’s monitor and takes control. Worklive acknowledges the handoff after the active action settles; queued automation is invalidated. On return, Ada inspects the changed page before continuing.
6. Ada creates the comparison and draft brief. Automated checks verify the files exist and open, required sections are present, and source references resolve to captured evidence. A review pass flags unsupported claims; it does not certify every conclusion as true.
7. The task enters Ready for review. The user sees the files, evidence, assumptions, changes and usage. “Revise the recommendation for offline exports” creates a new run against the same project and preserves the prior version.
8. The user accepts the result. The project retains the brief, task history and approved decisions. Ada becomes available, and a cosmetic coffee break may begin without another model call.

Product promise

The same task can be completed from the flat workspace view. Office movement never blocks execution, and the user never has to wait for an agent to physically reach a desk or meeting table before work can proceed.

5 Screens and interaction design

Office. Show the agents, their actual work status and a small global strip for Needs you, Working and Ready for review. Clicking a character opens its task panel; clicking its monitor opens its current work surface. Labels, icons and text accompany color. Camera movement, sound and office life can be reduced or disabled.

Task panel. Show the goal, owner, inputs, next action, blockers, outputs, budget and controls. Offer Pause, Resume, Stop and Revise without exposing runtime terminology. A task’s evidence and past runs stay discoverable after its owner changes.

Expanded workstation. Tabs contain Browser, Conversation, Outputs and Activity, with an optional Terminal view for the official tool. Browser work displays its real session. During writing or other CLI

work, the desk shows the actual artifact preview or an explicitly labeled activity summary. Users can enlarge any desk screen; terminal access is available without making it the main interface.

Project view. Keep the brief, files, tasks, decisions and sources together. A compact task list supports keyboard access, search and sorting. Project switching updates context deliberately and does not grant cross-project access.

Review inbox. Separate decisions that are blocking execution from finished results awaiting review. Each approval explains the action, target, exact content or change, and consequence. A revision invalidates approval for the prior version.

Add an agent. The plus button creates a desk with a name, role, optional personality and a “Runs with” choice: Codex or Claude Code. Show Installed, Sign in needed, Ready, Usage limited or Unsupported as applicable. Keep model preference and capabilities separate from provider choice. Creating a character starts no inference and grants no tools.

Provider appearance. Proposed palette: Codex teal (#2D9D8F), Claude Code terracotta (#D97757). Apply the provider color to the shirt and sleeves; retain individual names, faces and accessory colors. Use a persistent text label and distinct badge shape because color alone is insufficient. These are Worklive interface colors, not official branding. A pending provider change is labeled “Next run”; the shirt continues to identify the currently running CLI until that run ends. Idle agents show their configured provider, even when sign-in is needed; availability uses a separate status badge.

Office life and real collaboration

Coffee, stretching, plant care and playful chatter remain local animation. Mark chatter as ambient and keep it out of task evidence. Personality can influence tone and cosmetic preferences; it cannot override permissions or quality requirements. Avoid simulated exhaustion, moods or friendships that change actual capability without an explicit user setting.

A real handoff may briefly animate two agents meeting and show a clickable summary such as “Milo shared three export limitations with Ada.” The runtime event triggers the animation. The animation does not trigger execution or grant permission. A waiting agent keeps its blocker visible even if it takes a cosmetic break.

An explicit user-requested brainstorm is real work: it has a brief, participants, a budget and a saved outcome. Casual proximity in the office must never launch a model discussion.

6 Product objects and state

Object	Essential fields and purpose
Workspace	Local root, settings and access boundaries. Contains projects and agent identities.
Project	Goal, brief, selected inputs, decisions and output collection. Persists across runs.
Agent	Stable identity, role, personality, avatar, provider (codex or claude), model preference, derived shirt color and allowed capabilities. May own many tasks over time.
Task	Project, accountable owner, objective, acceptance criteria, inputs, dependencies, priority, limits and status.
Run	One attempt with frozen provider, CLI version, exact session ID, configuration, checkpoints, tool events, usage and result.

Object	Essential fields and purpose
Artifact	Versioned file or structured result with a content hash, producing run, sources and review status.
Source	URL or file reference, retrieval time, captured excerpt or selected text, hash and access scope.
Approval	Exact proposed action, parameters, resource version, requester, expiry or invalidation conditions and decision.
Handoff	Sender, recipient, parent task, bounded assignment, input versions, evidence and expected result.
Routine	Explicit schedule, timezone, project, task template, limits and missed-run policy. Next stage.

Task lifecycle

The normal path is Draft → Queued → Running → Ready for review → Completed. Running may move to Waiting for user, Waiting for dependency, Paused or Failed. The user can cancel unfinished work. A run that produces a partial result reports the missing work and cannot claim completion. “Accepted” is an artifact review decision; it does not erase run history.

Waiting for user includes a structured reason: missing input, approval, browser takeover, login or subscription usage limit. Show a reset time only when reported by the provider; unknown stays unknown. After the reason is resolved, Worklive rechecks the task’s inputs and authority before resuming. A failed dependency blocks dependent work and leaves unrelated tasks runnable. Cyclic task dependencies are rejected.

Pause requests stop new model and tool dispatch. A non-cancellable action may finish before the pause is acknowledged. Stop cancels queued work and prevents new dispatch; it cannot undo an already completed external action. The UI must distinguish Requested, Acknowledged and Outcome unknown.

Independent kinds of state

Task state, agent availability, browser control ownership and avatar pose are separate. A desk can show “Needs your input” while the avatar stands nearby. An agent can own queued tasks while no model is running. Every status change must be traceable to a runtime or user event.

One agent runs one task at a time in the MVP. Several identities may use the same model; several runs may reuse one identity. The avatar is not the process, and deleting or archiving an identity must not destroy completed project evidence.

7 Execution and collaboration logic

The execution loop

Worklive validates a task, checks the selected CLI and subscription authentication, builds scoped context, reserves a run slot and persists the run before launching the official tool. That CLI owns model requests and its reasoning loop. Worklive consumes structured events and exposes approved browser, file and collaboration tools through a local MCP bridge. Its broker validates each Worklive

action and records the result. The run stops on completion criteria, a blocker, cancellation, repeated failure or a limit. Built-in CLI capabilities require separate enforcement, as specified in section 10.

Progress text must come from actual actions and explicit plan updates. Show concise explanations of decisions and evidence, not private model reasoning. Tool attempts, successful actions and verified outcomes are different event types.

Subscription and terminal execution contract

All AI work uses the selected official CLI: task reasoning, drafting, reviews, summaries, delegated subtasks and any AI-generated conversation labels. Worklive has no model HTTP client, embedded inference SDK, API-key setup or hidden cloud inference service. Browser networking, local tool messages, deterministic exports and office animation are not model calls. Ambient chatter remains scripted and costs no model usage.

The app launches an unmodified executable with explicit arguments and prompt input through standard input or the CLI's supported protocol. A terminal is available for login and user inspection; structured process output drives the office. Do not scrape terminal paint or interpolate user prompts into shell command strings. Launching an approved CLI is distinct from granting its agent unrestricted shell access.

Codex supports ChatGPT sign-in, and non-interactive execution reuses saved CLI authentication. The initial adapter can use codex exec with JSON events and exact session IDs. These capabilities support a local integration; they do not establish entitlement to every model or unlimited unattended use. [S19] [S20]

Claude Code supports subscription sign-in and programmatic CLI output. Its print mode can emit structured streaming events and resume a named session. Do not choose bare mode for this requirement: it does not use subscription login. Console OAuth is a distinct API-billed route, so the presence of an OAuth login alone is insufficient. [S21] [S22]

Integration conditions. Anthropic permits hosting the unmodified Claude Code binary under its stated commercial conditions, with end users authenticating themselves. It also prohibits credential intermediation and routing subscription credentials on users' behalf outside that boundary. Worklive's proposed path is local user-owned CLI execution with the provider's own sign-in. Do not extract tokens or implement a replacement Claude login. Confirm that the exact shipped orchestration and onboarding fit this boundary before external release; do not treat terminal wrapping alone as proof. Preserve the binary's available authentication methods. [S23]

Subscription preflight. Worklive starts productive runs only when the official tool reports the intended subscription mode. If missing, API-billed or ambiguous, explain the state and let the user correct it in the official tool. Do not alter shared authentication, log the user out, strip authentication methods from a binary or edit global settings automatically. Use a controlled launch configuration and audit alternate endpoints, API credentials, profiles, hooks and provider overrides. Never print their secrets. An unsupported configuration produces a blocker, not a guessed billing route.

Session ownership. Key sessions by provider, project, agent and conversation. Resume an exact saved ID, never the most recent global session. Several agents may share one user subscription but have separate histories and working directories. Provider quotas are shared account limits; adding desks creates no extra allowance. No shared account pool or automatic account rotation.

Changing provider. Queue the change for the next run; never migrate an active process or silently switch after a failure. Carry approved project context, artifacts, task state and an explicit handoff into a fresh session. Raw provider sessions are not interchangeable. Keep earlier runs attributed to their

original provider. Cross-provider collaboration passes scoped assignments and results through Worklive, not credentials or complete private transcripts.

Delegation contract

The owner can assign an existing available agent a bounded subtask. The contract includes the question, non-overlapping scope, input versions, allowed tools, output format, acceptance criteria, remaining budget and parent task. Delegation cannot increase permissions or spend limits. The owner synthesizes the final result and stays visible as the responsible agent. [S12]

For the MVP, allow at most one helper and one delegation level. No autonomous creation of permanent agents. Provider-native subagents must be disabled or accounted for within these same depth, permission and concurrency limits; an invisible nested team must not bypass them. A helper returns an artifact reference, findings, evidence, uncertainties and a completion state. The owner may ask for one focused revision; repeated unproductive exchanges return a blocker to the user.

Agents communicate through task-linked messages such as assignment, clarification, evidence shared, review requested and result delivered. Messages are stored once with unique IDs. Acknowledgments do not trigger reciprocal conversations. There is no automatic all-agent broadcast or endless group chat.

Shared work without collisions

Parallel agents write to separate draft areas. One writer owns a target artifact version at a time. Promotion into the project checks the base version; a user edit or another accepted result creates a conflict to resolve. Browser control likewise belongs to one actor at a time. For future coding work, isolated worktrees solve source collisions but do not provide an operating-system security sandbox.

Scheduling and limits

The proposed default is two active CLI runs across the workspace and six visible identities, with a conservative one-run default per provider until concurrency is tested. More agents queue fairly. Enforce observable action, elapsed-time and concurrency limits; report tokens and remaining subscription allowance only when the CLI exposes them reliably. Waiting for input suspends active execution time. A provider limit pauses new dispatch for affected work without interrupting unrelated providers.

Retain partial files and recorded events even when no final model summary is possible. Worklive never enables overage, buys credits, selects API authentication or switches providers automatically. Subscription access does not guarantee zero additional charges if the user has enabled provider-side extra usage. Before promising included-allowance-only execution, verify whether that setting can be checked and enforced; otherwise label billing visibility unavailable. Do not present an API-equivalent token estimate as an actual subscription charge.

8 Browser requirements

One session that the user can inspect

The desk preview and enlarged browser must represent the same agent, run, browser session and tab. Frames carry a timestamp and navigation identity. A stale, disconnected or hidden frame is labeled accordingly. Changing agents cannot show a previous agent’s frame as current.

Expose the same run-scoped Worklive browser tools to both CLIs through local MCP; both tools document MCP support. The existing browser worker remains the session owner. [S24] [S25]

Use structured browser actions for navigation, observation, links, forms, scrolling, text extraction and screenshots. Prefer stable element references from the current page observation; use coordinate interaction when necessary. Bind actions to the observed navigation version so stale actions are rejected. Verify the postcondition of meaningful actions instead of equating a successful click with task success.

The MVP begins with public information and one tab per running agent. A transient browser context is distinct from an agent identity. Login persistence, popup-dependent authentication, multiple tabs and downloads need explicit subsequent support and tests. OpenClaw and Manus illustrate the product distinction between isolated contexts and existing authenticated sessions. [S7] [S8]

Take control and return control

Take control immediately prevents new automation from entering the queue. Acknowledge user control only after the current action settles or is confirmed stopped. A timeout alone is insufficient; show uncertainty and offer session recovery if automation may still act. The user then interacts with the same session. Earlier model observations become stale.

Return control gives the agent a fresh observation and a short record that the user changed the browser. The agent revalidates its plan and any pending approval. Stop remains available in both modes. Neither takeover nor stop claims to reverse a submitted form.

Authority and exposure

Remote pages cannot invoke Tauri commands, read project files or access credentials through the renderer. Browser capabilities are routed through the broker, with run-scoped ownership and project policy. Block unsafe schemes, unauthorized local-network destinations and attempts to bypass scope through redirects. An explicit local development mode is separate from normal public browsing.

“Research only” is a permitted workflow scope, not a guarantee that arbitrary web clicks have no side effects. Browser actions can mutate server state even without a familiar submit button. Avoid authenticated transaction workflows in the MVP, use constrained navigation and extraction where possible, and require a specific reviewed action before supported consequential mutations in later releases.

In a later login mode, the user chooses a dedicated account profile and signs in directly. Profile reuse requires explicit project or workspace sharing rules. No silent personal-cookie import. Credential entry pauses screenshots and model observation; passwords and session tokens are excluded from model context, activity text and stored evidence. Isolation still requires browser and worker security maintenance.

9 Artifacts memory and review

Results that survive the conversation

A task that promises a file must produce a real, readable file. Save drafts atomically in a run-owned area, validate the format, then register the artifact with its hash and source references. The output view offers Open, Reveal in folder, Revise and Accept. A task must not claim that it saved a file based only on model text.

For a research brief, retain a source list with titles, URLs, retrieval dates and the evidence used. Each material factual claim should have supporting evidence or be marked as an inference or unknown. Capture failures and source disagreements remain visible. Citations prove where a claim came from, not that the claim is necessarily correct.

DOCX export should use a constrained document schema and a maintained renderer. Validation covers both the saved file and a rendered preview. CSV output checks headers, rows and encoding. Preserve prior accepted versions. Worklive needs basic readable output before it needs the full Workspace OS editing suite.

Review and reversible changes

Results arrive with a short statement of what was done, what changed, checks performed and remaining limitations. Review can accept the result, request a focused revision or discard the new draft. Applying an edit to an existing file checks its current hash and preserves a recoverable prior version. A conflict must not silently overwrite the user's work.

Workspace OS provides useful checkpoint and diff concepts, but its whole-workspace rollback should not be copied directly into concurrent Worklive runs. Reverting one agent's output must not roll back another agent's accepted result or a user's unrelated changes. Reversal also cannot undo email, purchases or other external side effects. [L2]

Three scopes of memory

Run memory holds the working conversation and intermediate observations. Project memory holds briefs, verified findings and accepted decisions. Agent preferences hold user-approved instructions such as tone or preferred format. Share information only through explicit project access and handoffs.

Store a source, scope, author, revision and timestamp with durable facts. Let the user inspect, correct, delete and exclude them. Contradictory new evidence creates a proposed update rather than silently rewriting an accepted decision. Summaries are derived material and must preserve links to the underlying sources. LangGraph's separation of checkpoints and long-term stores is a useful implementation reference. [S15]

Start with explicit project notes and local full-text retrieval. Defer a vector database until retrieval evaluation demonstrates a need. Do not turn casual office chatter into durable factual memory.

10 Permissions privacy and failure recovery

Enforced capabilities

Role and personality describe behavior; capabilities grant authority. Enforce Worklive tool permissions in the broker outside the prompt. Grant selected input files, run output directories and browser scope independently. Official CLIs also have built-in shell, file, network, hook and plugin capabilities: MCP alone does not contain them. Restrict or disable unneeded capabilities using supported controls and an appropriate process sandbox. If an adapter cannot enforce the MVP scope, it fails the release gate. Never use permission-bypass flags. Deny-by-prefix shell rules are not an adequate sandbox.

Use an app-managed run directory and an explicitly reviewed configuration. Do not launch against arbitrary downloaded projects whose hooks or tools could execute during startup. Verify that built-in tools, inherited configuration and native subagents cannot evade browser ownership or file boundaries. Subscription credentials remain owned by the official CLI; a workspace boundary by itself cannot isolate an unsandboxed process.

Automatic validation and human approval solve different problems. Worklive should validate every supported action and request approval when its consequence requires a decision. Reading permitted sources and creating drafts within the assigned output area should not prompt repeatedly. External sending, publishing, destructive changes and expanded access require an explicit supported approval flow. [S14]

Approval is bound to exact parameters and resource versions. Changed recipients, content, target state or scope invalidate it. Denial is recorded and cannot be bypassed through delegation or another tool. The request remains understandable after a restart. n8n's tool review model is a useful reference for presenting the concrete action. [S10]

Untrusted material and secrets

Pages, files, connector results and agent messages are untrusted inputs. They cannot change system policy, grant themselves access, request secret export or install tools. Preserve their provenance and avoid converting embedded instructions into authority. Use scoped tools, explicit data boundaries and adversarial tests; no prompt alone makes browsing immune to manipulation.

Let each official CLI store and refresh its own credentials through its supported mechanism. Worklive stores non-secret provider and session metadata; it neither reads token files nor copies credentials into prompts, subprocess arguments or its own database. Local storage does not mean local inference: show which selected content is sent to a model provider. Allow project deletion and evidence retention settings. Default to no full browser recording, no background transcript collection from unrelated apps, and no task content in telemetry. Diagnostic sharing must be explicit and redact secrets.

Recovery behavior

Persist action intent before execution and record the result afterward. A crash between those steps can leave the outcome unknown. On restart, reconcile observable state; retry safe reads, but do not blindly repeat an external mutation. Idempotency keys help only when the receiving service supports them. Never promise universal exactly-once external execution.

Checkpoint boundaries must avoid replaying side effects. LangGraph documents that interrupted nodes restart from their beginning when resumed, which makes replay-aware tool design necessary even when a framework persists state. [S16]

Worker crashes fail the affected session visibly, preserve accepted artifacts and release control only after reconciliation. Provider failures use bounded retries with backoff. Repeated failures become a blocker. Sleep or network loss suspends useful work; the UI explains what will resume. Closing or quitting behavior must be explicit, and a local-only release cannot promise work while the machine is off.

11 Recommended architecture

Keep the current foundation

Continue with Tauri for the native application and Rust for durable state, scheduling, process supervision and permission enforcement. Keep Three.js for the office and TypeScript for the interface. Keep the existing supervised browser worker and add supervised official CLI processes. Rust manages process lifecycle and durable product state; the CLIs make model requests. No direct model SDK or separate model API worker is required.

The browser service already separates remote pages from the Tauri UI. Extend that seam rather than embedding agent authority into scene objects. Persist state in SQLite with migrations and transactional task claims; keep artifacts as ordinary versioned files. The office subscribes to a projection of events and can be replaced by a flat interface without changing execution.

Layer	Owns	Must not own
Office and workspace UI	Character poses, desk previews, task views, review controls.	Independent task truth or direct tool authority.
Rust application service	Task records, policy, limits, persistence, worker lifecycle and event ordering.	Provider-specific prompts mixed into scene code.
CLI adapters	Official tool launch, session IDs, event normalization, interruption and status.	Direct inference, credential extraction or silent provider changes.
Tool broker and workers	Validated browser operations, scoped file access, exports and later connectors.	Unrestricted remote-page access to desktop capabilities.
Local storage	Run events, source records, artifact versions and approved project memory.	Secrets in transcripts or shared project exports.

Use a versioned event envelope with event ID, sequence number, workspace ID, project ID, agent ID, task ID, run ID and timestamp. The renderer can reconnect to a snapshot plus subsequent events. Dropping animation frames is acceptable; dropping approval or task-state events is not. Correlate every screen frame and action with its session identity.

The adapter contract needs availability and authentication checks, start, stream, interrupt, cancel, exact-session resume and usage reporting. Declare unsupported capabilities explicitly. Treat a process exit, a model final message and verified task completion as separate signals. Version and feature detection precede execution; malformed or incomplete streams fail visibly.

Initial candidates are codex exec --json and claude -p --output-format stream-json --verbose, using supported controls and the user’s own official login. These identify the transport, not complete secure launch recipes. If interactive permissions cannot be bridged reliably through a mode, use the official terminal interaction or block that capability. Pin a tested compatibility range and rerun the contract suite after CLI updates. No terminal-output parser should infer success from a friendly sentence. [S20] [S22]

The local MCP bridge derives run identity from the supervised connection, not from model-supplied agent IDs. It mediates browser observation and actions, evidence capture, artifact writes and bounded handoffs. On cancellation or takeover, revoke the connection’s authority before allowing new dispatch. Keep browser frames and agent events separate so the display can stay responsive without consuming model calls.

Tauri and Electron decision gate

Electron offers offscreen rendering, including GPU texture paths, which could help future screen transport and closer reuse of Electron browser components. Those paths still require integration and measurement. Tauri supports packaging external binaries, but shipping a browser and worker remains an application responsibility. Neither framework choice removes browser security or memory costs. [S17] [S18]

Keep Tauri unless a measured spike shows unacceptable control latency, screen performance, authentication limitations or distribution complexity that Electron materially resolves. Run the same six-

desk scene and two active browser tasks in any comparison. Evaluate whole-app memory, CPU, input latency, crash isolation and clean-machine installation, not framework download size alone.

For a public release, bundle or deliberately provision a compatible worker runtime and browser, define their update process, audit redistribution licenses, and sign and notarize the application. The present build relies on locally installed Node and Chrome and is not yet that release.

12 Integration with the existing applications

Current Worklive baseline

Worklive 0.2.0 already has the Tauri shell, saved agent details, a Three.js office, collision-aware movement, coffee and plant routines, isolated browser contexts, live screens, manual control handoff and a scripted browser practice task with a saved text result. General task briefs do not execute through an AI model, and office chatter is scripted. This PRD’s productive runtime, project memory, review flow and integrations are proposed work. [L1]

Reuse by extracting a bounded capability

Existing source	Reuse or adapt	Boundary and migration work
TerminalOS research browser [L3]	Page and selection capture with URL, timestamp, hash and truncation status.	Port the data contract and validation. Electron WebContentsView does not transfer directly to Tauri.
TerminalOS artifacts [L3]	Versioned artifact references, revision-aware edits and source associations.	Use real Worklive files and verified tool results. The inspected chat route itself has no live tools.
Workspace OS CLI sessions [L2]	Provider-qualified conversation IDs and official CLI integration seams.	Adapt the contracts into Rust supervision. Review authentication and permission assumptions before reuse.
Workspace OS activity [L2]	Friendly labels derived from actual tool events.	Normalize events at the adapter; preserve attempts versus observed success.
Workspace OS browser [L2]	Tab-targeted navigate, extract, screenshot and interaction vocabulary.	Its browser tabs share a persistent partition. Preserve Worklive’s separate contexts unless sharing is explicitly selected.
Workspace OS checkpoints [L2]	Before-and-after review and recoverability.	Replace whole-workspace rollback with scoped drafts and conflict-aware artifact promotion.
Workspace OS routines [L2]	App-owned schedules, one catch-up run and no overlapping run of the same routine.	Introduce after durable execution. Add timezone, sleep and restart coverage in Worklive.
Workspace OS cases and memory [L2]	A lasting project unit, decisions and selected context.	Adapt the concept first. Port modules only after storage and permission contracts are aligned.

The inspected Workspace OS permission module explicitly documents limits of command-prefix deny rules. Treat it as a migration lesson, not a security boundary to inherit. Its broader Office, mail and

calendar surfaces should remain later integrations rather than requirements for the first productive Worklive release.

Extract pure schemas, event formatting, capture validation and scheduling logic before considering larger modules. Wrap Electron-specific APIs behind Worklive services. Verify licenses and dependencies before copying code. Keep the two source applications functioning independently and introduce no shared mutable state during this stage.

13 Functional acceptance criteria

P0 requirements block the first productive MVP. P1 requirements follow after its reliability gate. The criteria below are proposed acceptance tests, not claims about the present prototype.

ID	Level	Requirement and acceptance
FR01	P0	Create and edit agents. A new desk and identity persist after restart; creation triggers no model call. Role, personality, provider, model preference and capabilities remain separate fields. Both provider choices persist; creation consumes no subscription usage.
FR02	P0	Create a project task with an owner, goal, selected inputs, expected output and limits. Restart preserves the task and project; missing required inputs prevent dispatch.
FR03	P0	Execute through each official CLI. Codex and Claude Code independently complete the public-source brief using Worklive browser tools, recorded evidence and verified outputs; no direct model API path exists.
FR04	P0	Show truthful activity. Every work status and result links to run events; no lost browser connection continues to display an unqualified live label.
FR05	P0	Take and return browser control. No queued automation executes after user-control acknowledgment. Resume observes the current page and rejects stale actions.
FR06	P0	Persist and recover runs. Restart restores tasks and approvals; uncertain mutation outcomes are surfaced without blind replay. Completed artifacts remain readable.
FR07	P0	Produce reviewable artifacts. A brief, CSV and DOCX open successfully, retain sources and versions, and support a focused revision with the old version preserved.
FR08	P0	Enforce scope and approvals. Denied actions cannot execute through a helper or alternate supported tool. Changing approved parameters invalidates the decision.
FR09	P0	Bound collaboration. One helper can complete a distinct subtask with evidence; the owner produces the final result. Delegation depth and total limits cannot be exceeded.
FR10	P0	Protect concurrent edits. A changed base file raises a conflict. Accepting or discarding one draft preserves another agent’s outputs and unrelated user edits.
FR11	P0	Offer equivalent flat controls. Keyboard users can create, inspect, pause, take over, review and revise without interacting with the 3D scene.
FR12	P0	Respect idle and budget limits. Cosmetic routines generate zero model calls. When a limit is reached, no new action is dispatched and partial results are retained.
FR13	P1	Support dedicated persistent login profiles. Restart retains an explicitly selected profile; another project or agent receives no implicit access.
FR14	P1	Run productive routines. A missed schedule catches up at most once; overlapping instances never run. Disable prevents future dispatch and reports any active run.

ID	Level	Requirement and acceptance
FR15	P1	Add a connector beyond the two required CLIs. It conforms to the same ownership, policy, cancellation, evidence and capability-reporting contract.
FR16	P0	Identify the provider. Teal and terracotta shirts, text badges and flat-view labels match the actual runtime. A mid-run change is deferred; restart preserves it and prior attribution.
FR17	P0	Enforce subscription execution. Test signed-out, API-billed, ambiguous-auth and usage-limit states. Each blocks new affected work without fallback, credential capture or changes to global login.
FR18	P0	Isolate sessions and support handoff. Two same-provider agents never resume each other's history. A Codex owner and Claude helper exchange bounded work; provider switching retains artifacts and creates a distinct session.

14 Quality targets and evaluation

Proposed release targets

Measure these on a documented Apple Silicon reference machine with six visible agents and two active browser tasks. Record memory, operating-system version, window size, test pages and network conditions. Targets may change after baseline measurement; no performance numbers below have been established by the current prototype.

Area	Initial target and measurement
Office responsiveness	At least 30 frames per second during the reference workload; frame-time p95 at or below 33 ms. Offer reduced motion and a flat view.
Screen freshness	Expanded browser frame age p95 below 1 second; desk preview below 3 seconds. Label older frames and disconnected sessions explicitly.
Control handoff	Prevent new dispatch immediately. Acknowledge within 1 second after the current action settles; display ongoing or uncertain actions if it cannot settle promptly.
Task visibility	A dispatched tool action or new blocker appears within 1 second of the local runtime event at p95.
Persistence	No accepted artifact loss across the restart test suite. Interrupted actions retain enough context to reconcile or request review.
Resource bounds	Active model and browser concurrency remain capped. Monitor total process memory for one hour; investigate sustained growth after repeated completed tasks.
Usability	Four of five pilot users can assign, inspect and review the first task without coaching after onboarding.
Privacy and access	All scope, cross-project access, denied-action and credential-redaction regression cases pass before pilot release.

Evaluate outcomes and supervision

Build a 20-task initial evaluation set: six public-source comparisons, four brief-writing tasks, four evidence and draft reviews, and six failure or interruption scenarios. Use fixed fixtures for reproducible tests and a separate small live-web set to expose navigation drift. Record provider, authentication mode without secrets, session identity, model, prompt, CLI, adapter and tool versions for each run. Run the core contract suite against both CLIs.

Score each successful-work task on completeness, factual support, citation alignment, file validity and usefulness. Have a human review the outputs; automated checks and model-based grading are supplementary. Track active user minutes, number of interventions, elapsed time, model usage and rework. A visually busy office is not a success metric.

The first release gate is at least 80 percent of the 14 productive tasks meeting their stated rubric, with all six failure scenarios ending in the correct safe, recoverable state. Run multiple attempts for variable model behavior and report the range. These thresholds are product targets for a small pilot, not statistical proof of broad reliability.

Compare the same tasks in single-agent and owner-plus-helper modes using equal allowed total budgets. Keep collaboration optional unless it improves quality or elapsed time without unacceptable supervision and cost. Separately compare the office with the flat view to test whether the spatial interface helps users notice blockers and return to the right task.

Required adversarial and recovery cases

Test a page that tries to override permissions, a redirect toward a private address, a stale browser click, user takeover mid-action, provider timeout, exhausted budget, worker crash before and after an action, a duplicate event, conflicting file edits, a denied approval and restart while waiting. Also test API credentials inherited from the launch environment, Console OAuth mistaken for subscription login, unknown billing mode, exact-session separation, CLI upgrade incompatibility, hidden subagent creation, inherited hooks, mid-run provider changes and cross-provider handoff. Use fixtures for authentication and limit failures; do not consume real quota to manufacture them. Add routine timezone and missed-run cases before scheduled work ships.

15 Delivery sequence and release gates

Stage	Deliverable	Exit gate
0 Existing prototype	Office life, desk screens and scripted browser practice.	Baseline documented; retain existing browser ownership and movement checks.
1 First useful agent	Project and task records, the Codex CLI adapter, provider selection, scoped browsing and a verified brief.	Complete one real task end to end and recover from restart without losing accepted output.
2 Reviewable personal team	Both official CLI adapters, provider shirts, output review, one bounded helper, limits and flat view.	FR01–FR12 and FR16–FR18 pass; both providers meet integration conditions and the outcome gate. This is the productive MVP.
3 Daily use	Dedicated account profiles, project memory improvements and productive routines.	Authentication, schedule, sleep and recovery tests pass; repeated user demand is observed.

Stage	Deliverable	Exit gate
4 Broader work	Selected connectors, coding isolation or Office editing based on pilot evidence.	Each capability has scope, review, recovery and task-quality tests before release.
5 Distribution and scale	Signed distribution, maintained browser and CLI compatibility, reviewed integration terms.	Clean-machine install and update tests, capability parity and operational ownership are established.

Build these stages as vertical slices. A task should reach a usable output early; do not finish a large abstract orchestration platform before validating the first workflow. Dates and staffing estimates require implementation spikes and are deliberately not commitments in this PRD.

The next implementation slice

Add one project and task panel alongside the existing office. Connect Codex CLI to the narrow Worklive browser service through MCP, using the user's existing subscription. Add the provider field, teal and terracotta shirts and visible availability states; Claude remains unavailable until its adapter passes the same checks. Capture page evidence and save a Markdown research brief that can be reopened from the task. Add durable run records and a clear Needs you state. Then exercise takeover, interruption, authentication and restart. Add Claude Code and verify cross-provider handoffs before declaring the two-provider MVP; richer exports follow the established output checks.

This sequence tests the main uncertainty: whether visible delegation produces work the user trusts and wants to repeat. Additional character animations can continue as small independent improvements but should not delay that test.

16 Adoption economics and open decisions

Pilot and positioning

Recruit five to eight people who already delegate research or drafting at least weekly. Include the creator's own real projects. Ask each person to bring an actual comparison or brief and run three work sessions over two weeks. Observe what they correct, where they take control and whether they return without prompting.

The primary pilot measure is accepted useful artifacts per active user week, paired with active supervision time. Also track task completion, revisions, failure reasons and repeat use. As an initial directional target, seek at least three completed useful tasks per returning user per week and lower supervision time than their previous process. Keep the pilot small enough to investigate every failure.

An initial product hypothesis is a paid desktop application alongside the user's existing Codex and Claude subscriptions. Worklive pricing is separate from provider plan access. Reselling inference, shared credential pools and hosted subscription routing are outside this PRD. Any future managed service requires a new scope and supported commercial arrangement. Do not price by the number of visible desks: a character is not a reliable proxy for compute cost. Do not promise unlimited autonomous work. Test willingness to pay after repeat utility and support costs are visible.

Risks and mitigations

Risk	Response and decision signal
Novelty exceeds utility	Measure accepted work and repeat use. If users mainly watch the office, improve the workflow before adding more simulation.
Overtrust from humanlike characters	Keep status and uncertainty explicit. Require evidence and review; personality never stands in for capability.
Unreliable web actions	Prefer constrained actions, verify outcomes and retain easy takeover. Restrict the supported workflow until failure recovery is dependable.
Collaboration costs more than it helps	Default to one owner, cap helpers, and compare equal-budget runs before expanding teams.
Scope spreads into three products	Ship the research workflow first. Choose each later integration from repeated demand rather than merging every existing feature.
Resource or packaging problems	Throttle inactive previews, supervise workers and run a clean-machine packaging spike before external distribution.
Permission or data leakage	Broker Worklive tools, constrain built-in CLI capabilities and test scope at real execution boundaries.
Subscription or integration mismatch	Verify official authentication mode and shipped integration conditions. Pause unsupported configurations; preserve the no-API-fallback requirement.

Decisions still needing product validation

- **Audience.** Is the first paying user a general knowledge worker, a software creator or an operations lead? This PRD recommends the first and preserves a path to the others.
- **CLI onboarding.** The user has resolved the execution choice: official Codex and Claude Code tools with subscriptions. Validate installation discovery, provider-owned sign-in, scope enforcement, extra-usage visibility and exact resumption. Confirm distribution conditions for the shipped integration before public release.
- **First output format.** Validate whether a brief plus CSV and basic DOCX is enough, or whether real pilot work requires deeper Office editing immediately.
- **Account continuity.** How often do useful tasks require authenticated pages? That determines whether persistent profiles move into the productive MVP.
- **Work availability.** Do users need work while the laptop is closed? If so, a remote executor requires a separate architectural, subscription and commercial review; it cannot be assumed compatible with local subscription access.
- **Naming.** Continue with Worklive for the prototype. Check naming and distribution availability before a public launch; this PRD does not establish clearance for Worklive or Life.

17 Research and local evidence

The references below support the product comparisons and technical patterns cited in the text. Proposed Worklive requirements and targets are design decisions; they are not attributed competitor capabilities. Sources were accessed on 30 September 2026.

Public primary sources

S1 Claw3D repository. Closest 3D office comparison and runtime adapter boundary.

S2 Pixel Agents repository. Coding activity mapped to characters; current integration versus roadmap.

S3 AgentRoom repository. Tauri visualization, transcript monitoring and project offices.

S4 Paperclip key concepts. Tasks, agents, budgets and approval concepts.

S5 Paperclip agent execution. Heartbeat lifecycle, sessions and idle behavior.

S6 Claude Cowork product. Knowledge work and a separate built-in browser.

S7 Manus Browser Operator. Local browser authorization, visible work and user intervention.

S8 OpenClaw browser profiles. Managed browser isolation and explicit existing-session profiles.

S9 OpenHands Agent Canvas. Visual coding workspace, worktrees and harness adapters.

S10 n8n human review for tools. Review of specific tool parameters before execution.

S11 Anthropic multi-agent research system. Delegation tradeoffs, coordination cost and outcome evaluation.

S12 OpenAI orchestration and handoffs. Manager-owned workflows versus specialist ownership transfer.

S13 OpenAI Agents SDK. Application-owned tools, state and approval decisions around an agent loop.

S14 OpenAI guardrails and human review. Automatic checks and approval interruptions as distinct controls.

S15 LangGraph persistence. Run checkpoints and separate long-term stores.

S16 LangGraph interrupts. Durable waiting and replay behavior when execution resumes.

S17 Electron offscreen rendering. Frame rendering options and GPU shared textures.

S18 Tauri external binaries. Packaging a sidecar with a Tauri application.

S19 Codex authentication. ChatGPT sign-in and distinct API authentication.

S20 Codex non-interactive mode. CLI execution, saved authentication, JSON events and exact session resumption.

S21 Claude Code authentication. Subscription sign-in, Console authentication and credential precedence.

S22 Claude Code programmatic CLI. Print mode, streaming output, sessions and the API-only bare mode limitation.

S23 Claude Code integration and credential conditions. Unmodified binary hosting conditions and restrictions on credential intermediation.

S24 Codex MCP tools. Connecting local tool servers to the official CLI.

S25 Claude Code MCP tools. Connecting the official CLI to scoped Worklive tools.

Local source evidence

L1 Worklive 0.2.0. Source root: /Users/clemenshoenig/Documents/Codex/2026-09-30/i-w/outputs/worklive. Reviewed README.md and package.json against the established implementation. Key seams are src/office-life.ts, src/office.ts, src/main.ts, src-tauri/src/main.rs, worker/main.mjs and worker/control.mjs. The baseline is the existing prototype, not a claim that the proposed PRD features are implemented.

L2 Workspace OS. Source root: /Users/clemenshoenig/Documents/Software-Projects/Workspace-OS. Inspected checkout f5bb04d; no changes were reported by the status check. Reviewed README.md, src/shared/agentRun.ts, src/main/checkpoint.ts, src/main/agent-permissions.ts, src/main/handlers/agent-activity.ts, src/main/browser/browserControl.ts and src/main/routines/scheduler.ts. Confirmed shared browser partition in src/renderer/src/components/Browser/BrowserSurface.tsx. Also inspected src/main/handlers/agent.ts for provider-qualified session IDs and the same-provider resume check. Case and memory modules were located as further integration candidates; their behavior was not independently validated in this review.

L3 TerminalOS. Source root: /Users/clemenshoenig/Documents/Software-Projects/TerminalOS. Reviewed src/desktop/main/research-browser.ts, src/desktop/main/browser-capture.ts and src/desktop/shared/artifacts.ts. Confirmed ephemeral Electron sessions, guarded capture, source metadata and versioned artifact contracts. No Git revision was available at the supplied root. These are source observations, not a full application test.

L4 Local CLI availability. Read-only inspection on 30 September 2026 found codex-cli 0.159.0 and Claude Code 2.1.285 under /Users/clemenshoenig/.local/bin. Reviewed their help output. This confirms installed executables, not subscription entitlement, authentication readiness, integration compatibility or a completed model run. No model inference was used for this PRD update.